



ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR
Membre de
HONORIS UNITED UNIVERSITIES

EMSI ORANGERS

Département Génie Logiciel & Informatique

Casablanca — Maroc

RAPPORT DE PROJET ACADÉMIQUE

Module : Développement d'Applications Mobiles Android



SMARTLIFE

Conception et Realisation d'une Application de service Assistant Connecté piloté
par l'IA pour bien-être

 **Android / Kotlin**

 **FastAPI**

 **MongoDB**

 **Groq AI**

 **Firebase**

 **Présenté par :**

Mohammed Amine BENHADAR

Étudiant en Génie Logiciel

 benhadar.amine@etudiant.ac.ma

 **Encadré par :**

M. Mohammed BOUSMAH

Professeur — Département Informatique

 EMSI Orangers

Dédicaces

“

*À mes chers parents,
pour leur soutien moral et financier
inestimable,
leur patience et leur amour inconditionnels...*

*À mes enseignants et professeurs,
pour la transmission d'un savoir rigoureux
et leur dévouement envers leurs étudiants...*

*À tous mes amis et collègues de promotion,
pour les moments partagés et les
encouragements mutuels...*

*À l'innovation technologique,
source inépuisable de progrès humain et social.*

”

Mohammed Amine BENHADAR

Remerciements

Ce travail est le fruit de nombreuses collaborations et soutiens bienveillants.

Je tiens à exprimer ma profonde et sincère gratitude à **M. Mohammed BOUS-MAH**, mon encadrant pédagogique, pour ses conseils avisés, sa rigueur scientifique et sa disponibilité permanente tout au long de ce projet de développement mobile. Ses orientations ont été déterminantes dans la structuration et la réussite de ce travail.

Je remercie chaleureusement l'ensemble de l'**équipe pédagogique** du Département Génie Logiciel de l'EMSI Orangers, Casablanca pour la qualité de la formation dispensée et les bases solides qu'ils ont su me transmettre.

Mes remerciements vont également à mes **parents et à ma famille** pour leur soutien moral indéfectible, ainsi qu'à mes **camarades de promotion** pour les échanges constructifs et l'entraide dont ils ont fait preuve.

Enfin, je remercie les équipes des projets **open-source** (Android, OSMDroid, FastAPI) et les fournisseurs d'**API** (Groq, Firebase) dont les outils ont rendu ce projet possible.

Résumé

Le projet **SmartLife** s'inscrit dans l'évolution rapide de la santé numérique (*digital health*), visant à transformer le smartphone Android en un véritable compagnon de bien-être intelligent et personnalisé.

L'application repose sur une architecture **MVVM** (Model-View-ViewModel) qui garantit une séparation nette des responsabilités et facilite la maintenance du code. Pour le suivi sportif, un *Foreground Service* Android assure l'enregistrement GPS en continu, même lorsque l'application est en arrière-plan. Un **Coach IA** conversationnel, alimenté par l'API Groq (modèle LLaMA 3), fournit des conseils de santé personnalisés et répond aux requêtes vocales de l'utilisateur via la synthèse et la reconnaissance vocale.

Le backend, développé avec **FastAPI** (Python) et **MongoDB**, gère la persistance des données, la synchronisation des profils et génère des analyses de santé. L'authentification est sécurisée via **Firebase Auth**.

Mots-clés : **Android** **Kotlin** **Intelligence Artificielle** **GPS** **MVVM**
FastAPI **MongoDB** **Groq LLM**

Abstract

The **SmartLife** project addresses the growing field of digital health technologies, aiming to transform Android smartphones into intelligent, AI-powered wellness companions.

The application leverages a robust **MVVM** architecture ensuring clean separation between UI and business logic. A **Foreground Service** continuously tracks GPS data even in background mode. An AI-powered conversational coach, backed by the **Groq API** (LLaMA 3 model), delivers personalized health recommendations and processes voice queries using Speech-To-Text (STT) and Text-To-Speech (TTS) capabilities.

The backend, built with **FastAPI** and **MongoDB**, handles data persistence, profile synchronization, and health analytics generation. User authentication is managed securely through **Firebase Auth**.

Keywords : **Android** **Kotlin** **AI / LLM** **GPS Tracking** **MVVM** **FastAPI**
REST API

Liste des Abréviations

Abréviation	Signification complète
API	Application Programming Interface
	Global Positioning System
IA	Intelligence Artificielle
	JavaScript Object Notation
LLM	Large Language Model
	Model-View-ViewModel
NoSQL	Not only Structured Query Language
	Representational State Transfer
SDK	Software Development Kit
	Speech-To-Text
TTS	Text-To-Speech
	User Interface
UML	Unified Modeling Language
	User Experience
FCM	Firebase Cloud Messaging
	Integrated Development Environment

Table des matières

Dédicaces	1
Remerciements	2
Résumé	3
Abstract	4
Liste des Abréviations	5
Liste des Figures	8
Liste des Tableaux	9
Introduction Générale	10
1 Contexte Général du Projet	11
Introduction	11
1.1 Présentation du Projet	11
1.1.1 Problématique	11
1.1.2 Solution Proposée	12
1.2 Objectifs du Projet	13
1.3 Méthodologie et Planification	13
1.3.1 Méthodologie de Développement	13
1.3.2 Diagramme de Gantt	14
2 Analyse et Conception	15
Introduction	15
2.1 Étude Fonctionnelle	15
2.1.1 Analyse des besoins fonctionnels	16
2.1.2 Analyse des besoins non fonctionnels	17
2.2 Étude Conceptuelle	17
2.2.1 Règles de Gestion	17
2.2.2 Diagrammes UML	18

2.2.2.1	Diagramme de Cas d'Utilisation	18
2.2.2.2	Diagramme d'Activité	19
2.2.2.3	Diagramme de Classes	20
3	Étude Technique	23
	Introduction	23
3.1	Architecture du Projet Mobile (MVVM)	23
3.2	Infrastructure Backend (SmartLife API)	24
3.2.1	Serveur API avec FastAPI	25
3.2.2	Base de Données MongoDB	25
3.2.3	Service Intelligence Artificielle (Groq)	26
3.3	Stack Technologique Global	27
3.4	Environnement de Travail	28
4	Mise en Œuvre	29
	Introduction	29
4.1	Réalisation — Interfaces Utilisateur	29
4.1.1	Module d'Authentification	29
4.1.2	Module Profil Utilisateur	31
4.1.3	Dashboard Principal (Accueil)	32
4.1.4	Module Suivi GPS et Cartographie OSMDroid	34
4.1.5	Module Coach IA (Intelligence Artificielle)	36
4.1.6	Module Calendrier et Planification	38
4.1.7	Journal d'Activités	40
4.1.8	Paramètres et Personnalisation	41
4.2	Synthèse des Interfaces Réalisées	42
4.3	Intégration Backend et Communication	43
4.4	DevOps — Déploiement et Tests	43
4.4.1	Structure du Projet	44
4.4.2	Configuration et Déploiement	44
	Conclusion Générale et Perspectives	46

Table des figures

1.1	Diagramme de Gantt — Planification du projet SmartLife	14
2.1	Diagramme de Cas d’Utilisation de SmartLife	19
2.2	Diagramme d’Activité : Flux du Coach IA (Interaction Vocale)	20
2.3	Diagramme de Classes du Système SmartLife	21
3.1	Schéma de l’architecture MVVM appliquée à SmartLife	24
4.1	Module d’authentification — Connexion et création de compte	30
4.2	Gestion du profil utilisateur — Consultation et édition	31
4.3	Dashboard SmartLife — Indicateurs de santé et résumé du jour	33
4.4	Module GPS — Cartographie OSMDroid et tracé de parcours en temps réel	35
4.5	Module Coach IA — Interaction vocale et recommandations intelligentes	37
4.6	Module Calendrier — Planification hebdomadaire et ajout d’activités	39
4.7	Journal d’activités — Historique chronologique	40
4.8	Écran Paramètres — Personnalisation de l’expérience SmartLife	41

Liste des tableaux

1.1	Axes d'innovation de SmartLife	12
1.2	Objectifs fonctionnels et techniques de SmartLife	13
1.3	Phases de développement du projet SmartLife	14
2.1	Besoins fonctionnels détaillés de SmartLife	16
2.2	Besoins non fonctionnels de SmartLife	17
3.1	Détail des couches de l'architecture MVVM — SmartLife	24
3.2	Endpoints principaux de l'API FastAPI — SmartLife	25
3.3	Collections MongoDB — SmartLife	25
3.4	Stack technologique complet du projet SmartLife	27
3.5	Configuration de l'environnement de développement	28
4.1	Fonctionnalités implémentées — Module d'authentification	31
4.2	Métriques collectées lors du suivi GPS	36
4.3	Capacités du Coach IA implémentées dans SmartLife	38
4.4	Récapitulatif des écrans implémentés dans SmartLife	42
4.5	Flux de communication Mobile — Backend FastAPI	43
4.6	Organisation des répertoires du projet SmartLife	44

Introduction Générale

Ce rapport présente le projet **SmartLife**, développé dans le cadre du module de Développement d'Applications Mobiles Android à l'EMSI Orangers, Casablanca.

L'essor fulgurant des technologies mobiles et de l'intelligence artificielle a profondément transformé notre rapport à la santé et au bien-être. Aujourd'hui, le smartphone n'est plus un simple outil de communication, mais un compagnon quotidien capable de collecter des données biométriques, d'analyser nos habitudes et de nous guider vers un mode de vie plus sain.

Dans ce contexte, le projet **SmartLife** a pour ambition de développer un assistant de santé connecté, intelligent et personnalisé pour la plateforme Android. En combinant la puissance du suivi GPS en temps réel, l'intelligence artificielle conversationnelle (via les modèles LLM de Groq), et une interface utilisateur moderne conforme aux standards Material 3, SmartLife vise à offrir une expérience de bien-être complète et proactive.

Ce rapport est structuré en **quatre chapitres** complémentaires :

Chap. 1 Contexte Général du Projet : Présentation des problématiques identifiées, de la solution proposée et de la planification.

Chap. 2 Analyse et Conception : Étude fonctionnelle, règles de gestion et modélisation UML.

Chap. 3 Étude Technique : Architecture MVVM, stack technologique et infrastructure backend.

Chap. 4 Mise en Œuvre : Réalisation des interfaces, intégration de l'IA et déploiement DevOps.

Contexte Général du Projet

Introduction

Ce chapitre pose les fondations du projet SmartLife en décrivant le contexte d'émergence de l'application, les problèmes qu'elle entend résoudre, et la vision stratégique qui guide son développement. Une planification rigoureuse, illustrée par un diagramme de Gantt, complète cette présentation.

1.1 Présentation du Projet

SmartLife est une application mobile Android intelligente dédiée à l'amélioration des habitudes de vie grâce à l'intelligence artificielle. Elle s'adresse à toute personne souhaitant suivre ses activités sportives, bénéficier de conseils personnalisés et organiser son bien-être quotidien depuis son smartphone.

L'application cible un marché en pleine expansion : celui de la **santé numérique** (*mHealth*). Selon les études récentes, plus de 50% des utilisateurs de smartphones ont téléchargé au moins une application de santé, mais seule une minorité les utilise sur le long terme, en raison du manque de personnalisation et d'interactivité.

1.1.1 Problématique

Les applications de santé existantes présentent plusieurs lacunes significatives qui freinent leur adoption durable par les utilisateurs.

- P1. Manque d'intelligence :** Les applications classiques se contentent d'afficher des données brutes (pas, calories, distance) sans fournir de recommandations intelligentes adaptées au profil et à l'historique de l'utilisateur.
- P2. Discontinuité du suivi :** Les utilisateurs ont des difficultés à maintenir un suivi continu de leurs activités physiques, notamment lors de sessions longues où l'application peut être interrompue par le système.

- P3. Interfaces peu engageantes :** Les interfaces actuelles manquent souvent d’ergonomie, d’esthétique moderne et d’interactivité, ce qui conduit à un abandon rapide de l’application.
- P4. Fragmentation des outils :** Les utilisateurs doivent jongler entre plusieurs applications (tracker GPS, agenda, chatbot de santé), sans synchronisation entre elles.

1.1.2 Solution Proposée

SmartLife répond à ces défis par une approche intégrée et intelligente, articulée autour de quatre axes d’innovation :

TABLE 1.1 – Axes d’innovation de SmartLife

Composante	Technologie	Valeur Ajoutée
Coach IA	Groq API (LLaMA 3)	Conseils santé personnalisés, interaction vocale naturelle
Suivi GPS	Android Location API	Tracking en temps réel, résistant aux interruptions
Interface	Material Design 3	Ergonomie moderne, mode sombre/clair adaptatif
Cartographie	OSMDroid	Visualisation des parcours sur carte libre OpenStreetMap
Planification	Calendrier intelligent	Emploi du temps hebdomadaire synchronisé avec le cloud
Backend	FastAPI + MongoDB	Persistance fiable, synchronisation temps réel

1.2 Objectifs du Projet

TABLE 1.2 – Objectifs fonctionnels et techniques de SmartLife

N°	Objectif	Description	Priorité
O1	Authentification sécurisée	Connexion/inscription via Firebase Auth avec persistance de session	Critique
O2	Suivi GPS continu	Enregistrement du parcours en Foreground Service même hors-app	Critique
O3	Coach IA conversationnel	Dialogue vocal/textuel avec l'API Groq pour des conseils santé	Haute
O4	Cartographie dynamique	Affichage et enregistrement des parcours sur OpenStreetMap	Haute
O5	Calendrier intelligent	Planification et gestion des séances sportives hebdomadaires	Moyenne
O6	Historique & statistiques	Visualisation de la progression avec graphiques et rapports	Moyenne
O7	Notifications push	Rappels et encouragements via Firebase Cloud Messaging	Basse

1.3 Méthodologie et Planification

1.3.1 Méthodologie de Développement

Le projet a suivi une approche **Agile itérative**, découpée en sprints de deux semaines. Cette méthode permet une adaptation continue aux retours utilisateurs et une livraison progressive des fonctionnalités.

TABLE 1.3 – Phases de développement du projet SmartLife

Phase	Nom	Activités principales	Durée
1	Analyse & Conception	Étude des besoins, modélisation UML, architecture	2 semaines
2	Setup & Architecture	Configuration du projet Android, MVVM, structure des packages	1 semaine
3	Authentification	Firebase Auth, écrans Login/Register, gestion de session	1 semaine
4	Suivi GPS	GpsTrackingService, OSMDroid, calcul des métriques	2 semaines
5	Coach IA	Intégration Groq API, STT/TTS, interface de chat	2 semaines
6	Calendrier	CRUD des tâches, synchronisation backend, UI calendrier	1 semaine
7	Backend FastAPI	API REST, endpoints, MongoDB, Firebase Admin	2 semaines
8	Tests & Déploiement	Tests sur appareil réel, corrections de bugs, documentation	1 semaine

1.3.2 Diagramme de Gantt

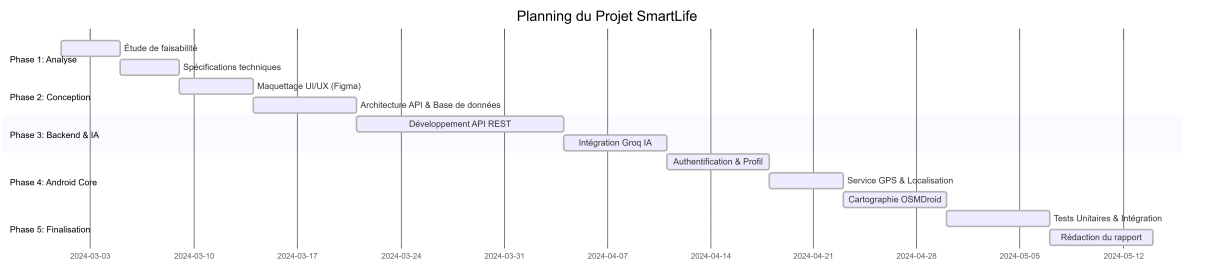


FIGURE 1.1 – Diagramme de Gantt — Planification du projet SmartLife

Conclusion

Ce premier chapitre a permis de contextualiser le projet SmartLife, d’en définir les problématiques et d’en préciser les objectifs. La planification par phases assure une progression méthodique du développement. Le chapitre suivant se concentre sur la phase d’analyse et de conception, qui traduit ces ambitions en spécifications techniques rigoureuses.

Analyse et Conception

Introduction

La phase d'analyse et de conception constitue la colonne vertébrale de tout projet de développement logiciel. Elle permet de traduire les idées abstraites en spécifications techniques rigoureuses. Dans ce chapitre, nous explorons en détail les dimensions fonctionnelles et conceptuelles de **SmartLife** : nous commençons par une étude des besoins pour définir *ce que* le système doit faire, avant de passer à une étude conceptuelle qui modélise *comment* il sera structuré à l'aide du langage UML (Unified Modeling Language).

2.1 Étude Fonctionnelle

2.1.1 Analyse des besoins fonctionnels

TABLE 2.1 – Besoins fonctionnels détaillés de SmartLife

BF#	Fonctionnalité	Description détaillée	Acteur
BF1	Authentification	Création de compte sécurisé, connexion et gestion du profil utilisateur via Firebase Auth	Utilisateur
BF2	Suivi GPS en temps réel	Enregistrement du parcours via Foreground Service, calcul de distance, durée et tracé cartographique	Utilisateur / Système Android
BF3	Coach IA conversationnel	Module de chat vocal/textuel, analyse sémantique des données, génération de conseils personnalisés	Utilisateur / Groq AI
BF4	Planification d'activités	Calendrier hebdomadaire pour programmer des séances sportives avec rappels	Utilisateur
BF5	Historique & statistiques	Archivage des performances, visualisation de la progression via graphiques	Utilisateur
BF6	Visualisation cartographique	Affichage des parcours passés sur une carte OpenStreetMap interactive	Utilisateur
BF7	Notifications intelligentes	Rappels push personnalisés basés sur la routine planifiée	Utilisateur / FCM

2.1.2 Analyse des besoins non fonctionnels

TABLE 2.2 – Besoins non fonctionnels de SmartLife

Critère	Exigence	Indicateur cible
Performance	Latence de réponse du Coach IA minimale, interface fluide à 60 fps	< 2 secondes
Efficacité énergétique	Optimisation du Foreground Service GPS pour préserver la batterie	Drain < 5%/h
Ergonomie UI/UX	Conformité Material Design 3, mode sombre/clair, navigation intuitive	Score UX > 8/10
Sécurité	Chiffrement des données de localisation, authentification Firebase	HTTPS + Auth token
Disponibilité	Backend accessible en permanence avec gestion des erreurs réseau	Uptime > 99%
Maintenabilité	Architecture MVVM, code documenté, commits atomiques Git	Couverture doc > 80%

2.2 Étude Conceptuelle

2.2.1 Règles de Gestion

Règles métier du système SmartLife

- RG1. Authentification obligatoire :** Un utilisateur doit obligatoirement être authentifié via Firebase Auth pour accéder à toutes les fonctionnalités sensibles (sauvegarde, Coach IA, historique).
- RG2. Précision GPS :** Le calcul de la distance ne démarre que si la précision du signal GPS est inférieure à un seuil de 20 mètres. En-deçà, un avertissement est affiché.
- RG3. Contexte du Coach IA :** Les conseils du Coach IA sont générés prioritairement sur la base de la dernière activité enregistrée et du score de discipline calculé.
- RG4. Intégrité des tâches :** Une tâche planifiée dans le calendrier doit obligatoirement contenir au moins un titre, une date et une heure de début valides.

- RG5. Tri chronologique :** L'historique des activités est systématiquement trié par ordre antéchronologique (la plus récente en premier) pour faciliter la consultation.
- RG6. Unicité du profil :** Un utilisateur ne peut posséder qu'un seul profil, lié à son identifiant Firebase (UID). Toute duplication est gérée côté backend.
- RG7. Format des données GPS :** Les coordonnées GPS sont stockées au format GeoJSON dans MongoDB pour permettre des requêtes géospatiales futures.

2.2.2 Diagrammes UML

Pour garantir une architecture solide et communicable, nous avons utilisé trois diagrammes UML complémentaires, chacun apportant une perspective différente sur le système.

2.2.2.1 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation (Figure 2.1) délimite le périmètre fonctionnel du système SmartLife et identifie les interactions entre les acteurs et le système.

Acteurs identifiés :

- **Utilisateur (acteur principal)** : personne physique qui interagit directement avec l'application via son smartphone.
- **Système Android (GPS)** : acteur secondaire fournissant les données de localisation via les APIs système.
- **Groq AI Service** : acteur secondaire externe qui traite les requêtes linguistiques et génère les réponses du Coach IA.

Points clés : L'authentification conditionne l'accès à la plupart des fonctionnalités (relation *include* implicite). Le démarrage du suivi GPS implique une interaction avec le Système Android. La discussion avec le Coach IA engage le service Groq AI externe.

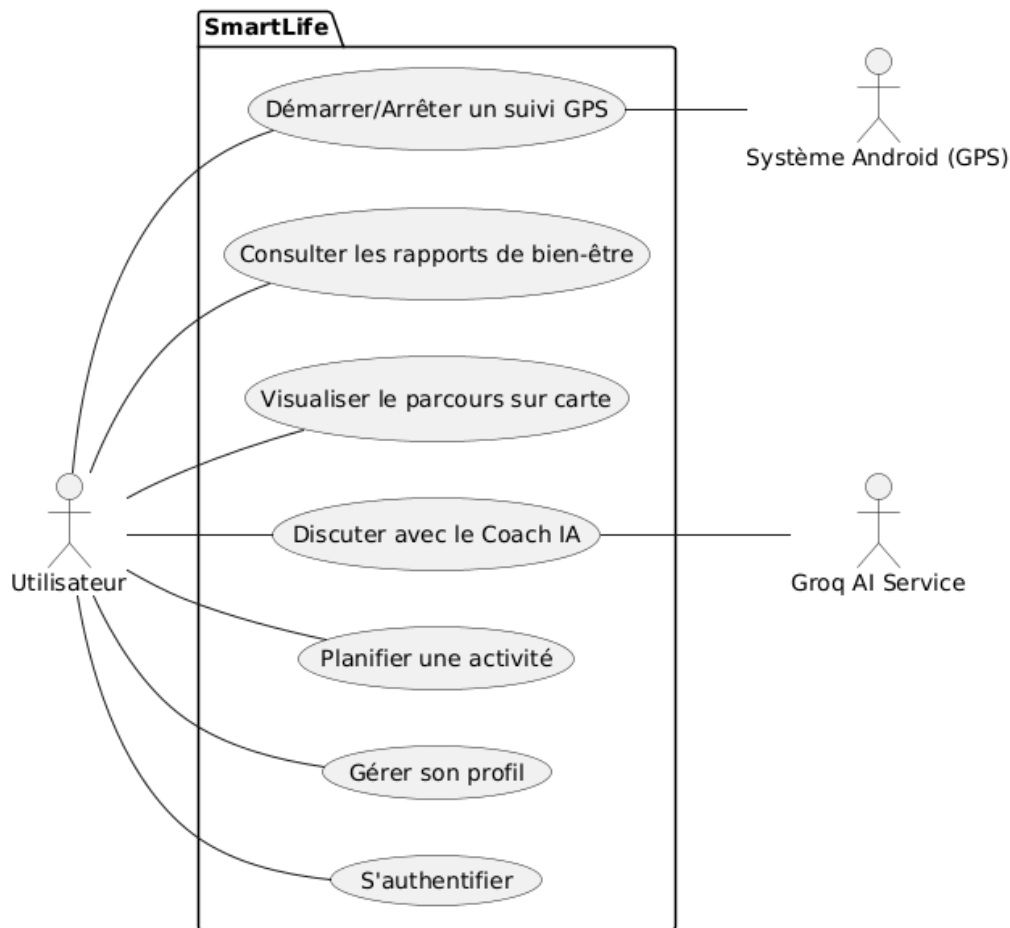


FIGURE 2.1 – Diagramme de Cas d'Utilisation de SmartLife

2.2.2.2 Diagramme d'Activité

Le diagramme d'activité (Figure 2.2) modélise de façon détaillée le flux dynamique de l'interaction vocale entre l'utilisateur et le Coach IA.

i Analyse du Diagramme d'Activité

Flux principal — Interaction Vocale avec le Coach IA :

1. L'utilisateur ouvre l'onglet "Coach IA" et appuie sur le bouton micro (FloatingActionButton).
2. Le service `SpeechRecognizer` d'Android est activé.
3. **Décision 1** : Signal audio détecté ?
 - *Non* → Affichage du message "Aucune voix détectée", retour au début.
 - *Oui* → Conversion voix-texte (STT) et affichage de la requête.
4. La requête est envoyée au backend via l'API Groq.
5. Le modèle LLM procède à l'analyse contextuelle.
6. **Décision 2** : Réponse générée avec succès ?

- *Non* → Affichage d'un message d'erreur réseau.
- *Oui* → Affichage de la réponse textuelle + animation avatar + lecture TTS.

Scénario : Interaction Vocale avec le Coach IA



FIGURE 2.2 – Diagramme d'Activité : Flux du Coach IA (Interaction Vocale)

2.2.2.3 Diagramme de Classes

Le diagramme de classes (Figure 2.3) représente la structure statique du domaine métier et les relations entre les entités principales du système.

i Analyse du Diagramme de Classes

Entités et relations :

- **User** : classe pivot centrale. Possède un `uid`, `firstName`, `lastName`, `email`, `birthDate`. Expose les méthodes `register()` et `login()`.
- **Activity** (agrégation 0..*) : enregistre les sessions sportives avec `type`, `dis-`

tance, durée et date. Un utilisateur peut avoir plusieurs activités.

- **AICoach** (association 0..*) : traite les requêtes via `getInsights(userId)` et `processChat(message)`. Un utilisateur peut avoir plusieurs sessions de coaching.
- **Schedule** (composition 1) : emploi du temps unique par utilisateur, composé de tâches.
- **Task** (agrégation 0..*) : chaque tâche planifiée contient un identifiant, une heure, un type d'activité, une localisation et une durée.

Multiplicités : La composition $\text{User} \rightarrow \text{Schedule}$ (1 à 1) garantit l'unicité. Les agrégations (1 à 0..*) reflètent la flexibilité du suivi.

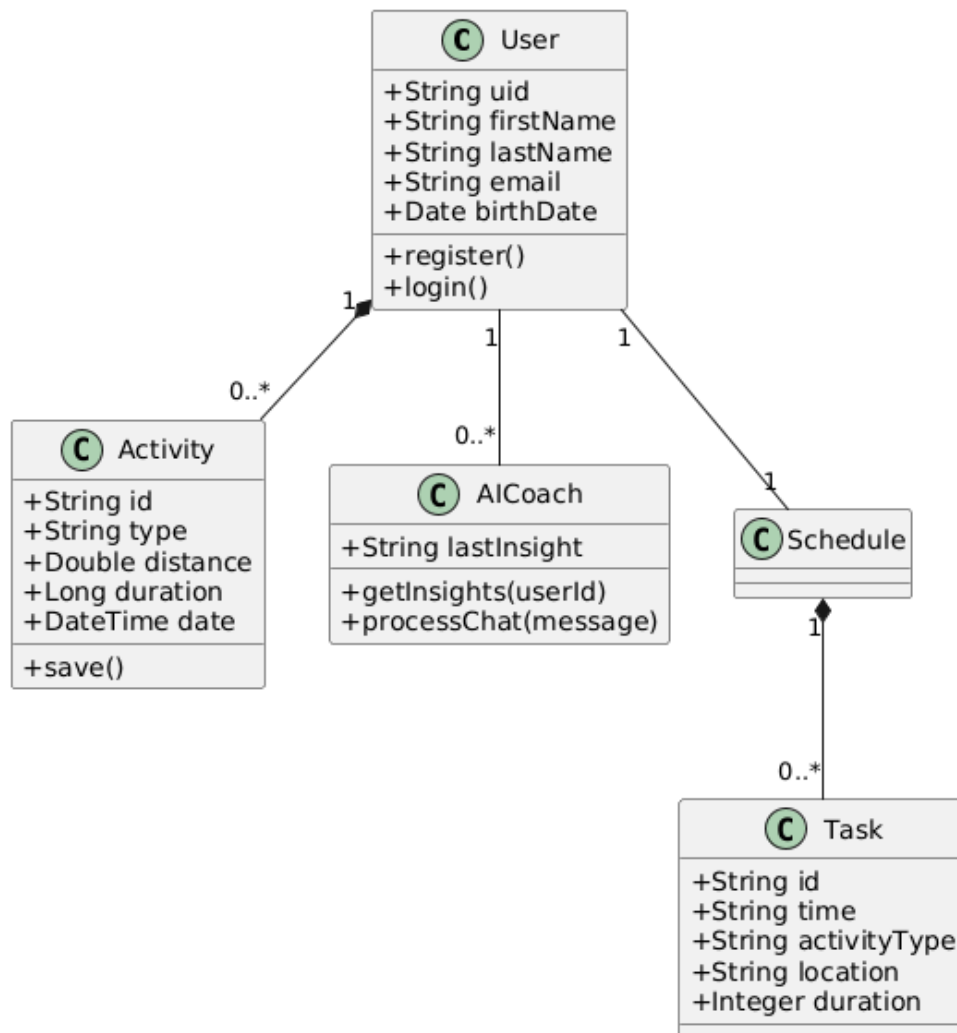


FIGURE 2.3 – Diagramme de Classes du Système SmartLife

Conclusion

Ce chapitre a permis de passer d’une vision globale du projet à une structuration technique précise. En définissant des besoins fonctionnels et non fonctionnels clairs, en formalisant les règles de gestion et en modélisant le système via trois diagrammes UML complémentaires, nous avons établi une feuille de route fiable pour la phase de réalisation. Cette conception garantit que l’application **SmartLife** sera non seulement complète, mais aussi évolutive et robuste.

Étude Technique

Introduction

La concrétisation de **SmartLife** repose sur une synergie entre une architecture logicielle moderne et des technologies de pointe. Ce chapitre détaille les choix techniques qui garantissent la robustesse, la fluidité et l'intelligence du système : l'organisation du code via l'architecture MVVM, l'infrastructure backend FastAPI/MongoDB, et les services critiques comme le GPS et l'IA.

3.1 Architecture du Projet Mobile (MVVM)

Pour structurer le code Android et faciliter sa maintenance, nous avons adopté l'architecture **MVVM (Model-View-ViewModel)**, recommandée par Google et l'Android Architecture Components Guide.

TABLE 3.1 – Détail des couches de l'architecture MVVM — SmartLife

Couche	Composants Android	Rôle & Responsabilités
Vue (View)	Activities, Fragments, XML Layouts	Affichage exclusif. Observe les LiveData du ViewModel via Data Binding. Ne contient aucune logique métier.
ViewModel	AndroidViewMod LiveData, StateFlow	Noyau décisionnel. Prépare les données, gère les états (Loading, Success, Error). Survit aux rotations d'écran.
Repository	Repository classes	Façade de données. Centralise et arbitre entre les sources locales (Room, SharedPrefs) et distantes (Retrofit).
Source de données locale	Room, Shared-Preferences	Persistance locale. Cache des données pour le mode hors-ligne (prévu).
Source de données distante	Retrofit 2, OkHttp	Communication réseau asynchrone avec le backend FastAPI.
Services d'arrière-plan	GpsTrackingServi (Foreground)	Maintien du suivi GPS même si l'app est réduite ou l'écran éteint.

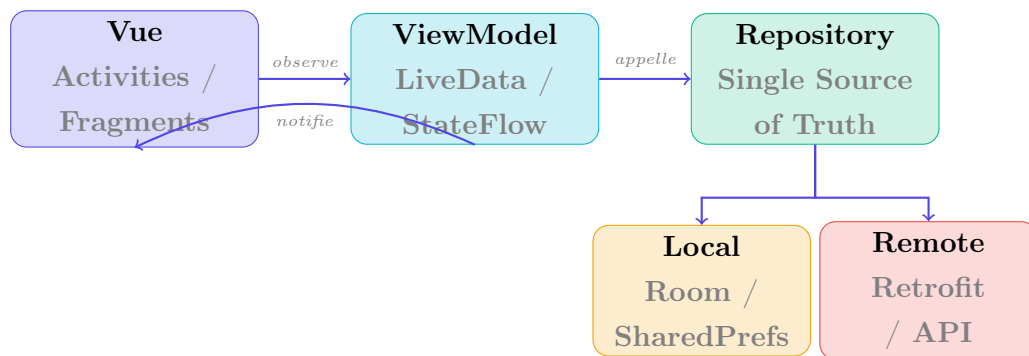


FIGURE 3.1 – Schéma de l'architecture MVVM appliquée à SmartLife

3.2 Infrastructure Backend (SmartLife API)

3.2.1 Serveur API avec FastAPI

TABLE 3.2 – Endpoints principaux de l’API FastAPI — SmartLife

Endpoint	Méthode	Description	Auth
/users/{uid}	GET	Récupération du profil utilisateur complet	✓
/users/{uid}	PUT	Mise à jour des informations du profil	✓
/activities/	POST	Soumission d’une nouvelle activité GPS	✓
/activities/{uid}	GET	Historique des activités de l’utilisateur	✓
/schedule/{uid}	GET	Récupération de l’emploi du temps hebdomadaire	✓
/schedule/	POST	Création/mise à jour d’une tâche planifiée	✓
/ai/insights/{uid}	GET	Génération d’insights santé via Groq AI	✓
/ai/chat/	POST	Envoi d’un message au Coach IA	✓

3.2.2 Base de Données MongoDB

MongoDB a été choisi pour sa flexibilité avec les données non structurées (traces GPS, recommandations IA) et son pilote asynchrone Motor, parfaitement compatible avec FastAPI.

TABLE 3.3 – Collections MongoDB — SmartLife

Collection	Données stockées	Index
users	uid, firstName, lastName, email, birthDate, createdAt	uid (unique)
activities	userId, type, distance, duration, coordinates (GeoJSON), startTime	userId, date
schedules	userId, weekYear, tasks[]	userId
ai_conversations	userId, messages[], createdAt	userId, date

3.2.3 Service Intelligence Artificielle (Groq)

Le backend intègre un module dédié (`ai_service.py`) interfacé avec l'API **Groq** utilisant le modèle **LLaMA 3 70B**. Ce service :

- Analyse le contexte de l'utilisateur (dernière activité, emploi du temps, score de discipline).
- Génère des *insights* de santé personnalisés (score de discipline, conseils nutritionnels, encouragements).
- Fournit des réponses en temps réel au Coach IA conversationnel.

3.3 Stack Technologique Global

TABLE 3.4 – Stack technologique complet du projet SmartLife

Catégorie	Technologie	Justification du choix
Langage mobile	Kotlin 1.9	Langage officiel Android, null-safety, coroutines natives
Architecture	MVVM + Repository	Séparation des responsabilités, testabilité, survie aux rotations
Communication réseau	Retrofit 2 + OkHttp	Client HTTP type-safe pour Kotlin/Android, support coroutines
Cartographie	OSMDroid	OpenStreetMap libre, pas de quota API, support offline
Authentification	Firebase Auth	Gestion sécurisée multi-provider (email, Google)
Notifications push	Firebase FCM	Intégration native Android, faible latence
Backend framework	FastAPI (Python)	Performances asynchrones, documentation auto-générée (Swagger)
Base de données	MongoDB + Motor	Flexibilité NoSQL, données géospatiales, scalabilité horizontale
IA / LLM	Groq API (LLaMA 3)	Inférence ultra-rapide (<1s), modèle de langage de pointe
Reconnaissance vocale	Android SpeechRecognizer	API native, support multilingue, sans coût supplémentaire
Synthèse vocale	Android TextToSpeech	API native, personnalisable (vitesse, pitch, langue)
IDE mobile	Android Studio + Hedgehog	Émulateur intégré, Logcat, profiler de performances
IDE backend	VS Code + Python ext.	Léger, excellent support Python, débogage intégré
Versionnement	Git + GitHub	Historique des commits, branches de fonctionnalités

3.4 Environnement de Travail

TABLE 3.5 – Configuration de l’environnement de développement

Paramètre	Valeur
Système d’exploitation	Windows 11 / macOS (développement)
Android Studio	Hedgehog (2023.1.1)
SDK Android minimum	API 26 (Android 8.0 Oreo)
SDK Android cible	API 34 (Android 14)
Python version	3.11+ (backend FastAPI)
MongoDB version	7.0 (Cloud Atlas / local)
Appareil de test	Smartphone réel + émulateur Pixel 6 API 34
Connexion test	Réseau Wi-Fi local (serveur sur 0.0.0.0:8000)

Conclusion

L’étude technique confirme que l’application **SmartLife** est un système complet où le mobile Android et le backend Python collaborent étroitement via une API REST. L’architecture MVVM côté client garantit une interface réactive et maintenable, tandis que le backend FastAPI/MongoDB assure la persistance et l’intelligence des données. L’intégration du modèle LLaMA 3 via Groq offre une expérience conversationnelle de qualité, différenciant SmartLife des applications de santé classiques.

Mise en Œuvre

Introduction

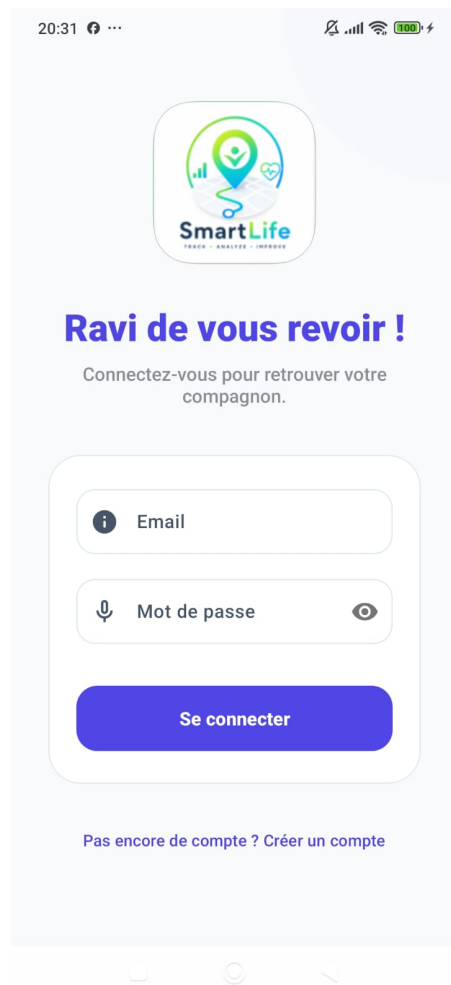
Après avoir défini l'architecture et les technologies, ce chapitre présente la phase concrète de réalisation de **SmartLife**. Nous illustrons ici l'ensemble des interfaces finales de l'application à travers des captures d'écran réelles prises sur appareil physique, accompagnées de descriptions détaillées de chaque fonctionnalité implémentée. Nous décrivons ensuite les processus DevOps qui ont permis de rendre le système opérationnel.

4.1 Réalisation — Interfaces Utilisateur

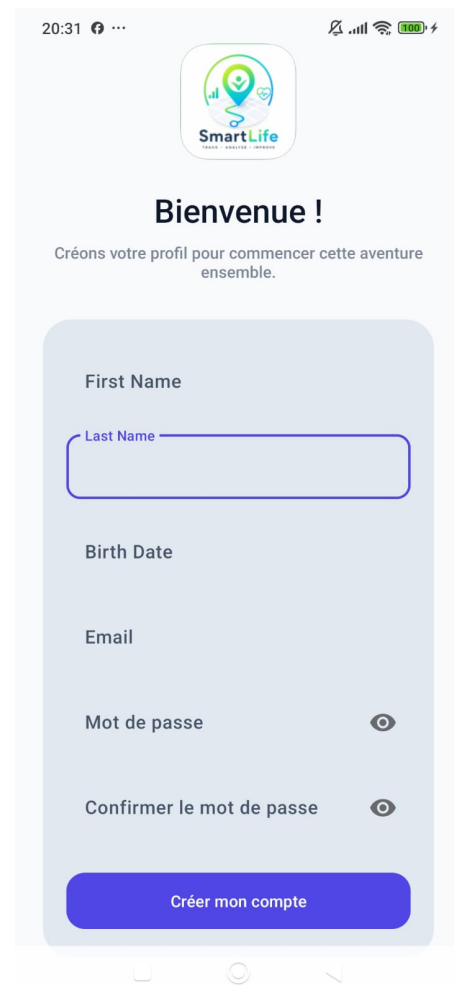
L'interface de SmartLife a été conçue selon les directives **Material Design 3** de Google, avec une palette de couleurs cohérente (violet indigo / cyan), une barre de navigation inférieure à cinq onglets, et un mode sombre/clair configurable.

4.1.1 Module d'Authentification

La gestion des sessions utilisateurs est assurée par **Firestore Authentication**. L'application propose deux flux distincts : la connexion pour les utilisateurs existants et l'inscription pour les nouveaux comptes. La validation des champs s'effectue en temps réel avant tout envoi au serveur.



(a) Écran de connexion



(b) Écran d'inscription

FIGURE 4.1 – Module d'authentification — Connexion et création de compte

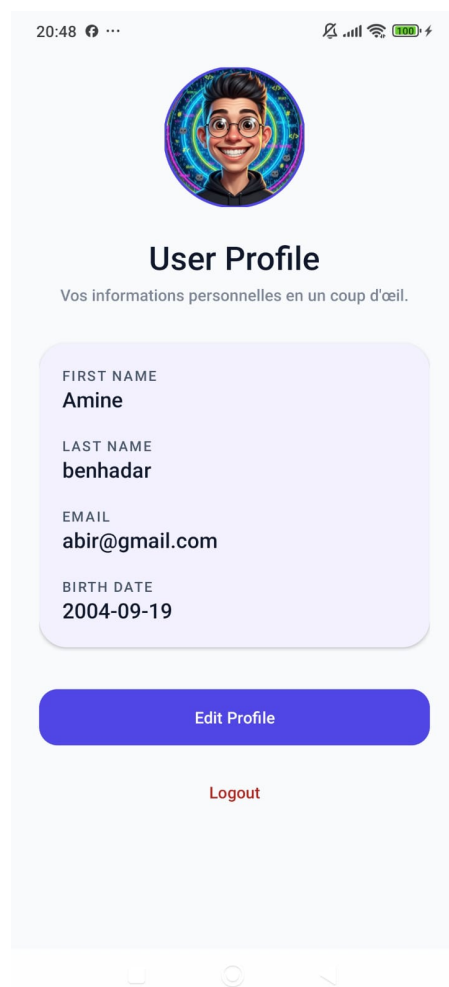
Description des écrans :

- **Écran de connexion (Figure 4.1a)** : Affiche le logo SmartLife avec le slogan "*Track · Analyze · Improve*". Le formulaire comprend deux champs (email et mot de passe avec affichage/masquage) et un bouton "**Se connecter**" en violet. Un lien de redirection vers l'inscription est disponible en bas. L'authentification est gérée par Firebase Auth avec persistance de la session.
- **Écran d'inscription (Figure 4.1b)** : Formulaire complet de création de compte avec les champs : Prénom, Nom, Date de naissance, Email, Mot de passe et Confirmation du mot de passe. Un bouton "**Créer mon compte**" valide et envoie les données à Firebase puis au backend FastAPI.

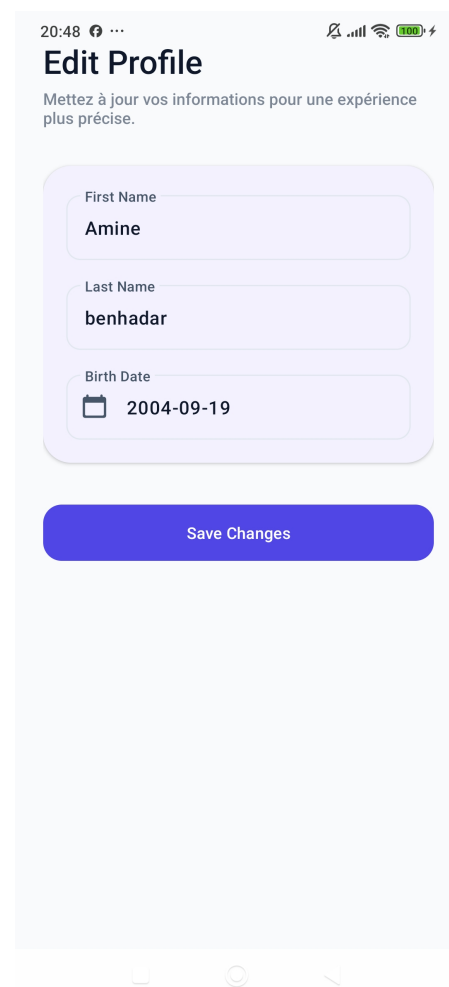
TABLE 4.1 – Fonctionnalités implémentées — Module d’authentification

Fonctionnalité	Détails d’implémentation	Statut
Connexion email/password	Firestore Auth, persistance via SharedPreferences	✓ OK
Inscription complète	Validation champs + création profil MongoDB via FastAPI	✓ OK
Affichage/masquage password	Toggle avec icône œil sur les champs sensibles	✓ OK
Redirection automatique	Si session active → Dashboard, sinon → Login	✓ OK

4.1.2 Module Profil Utilisateur



(a) Consultation du profil



(b) Modification du profil

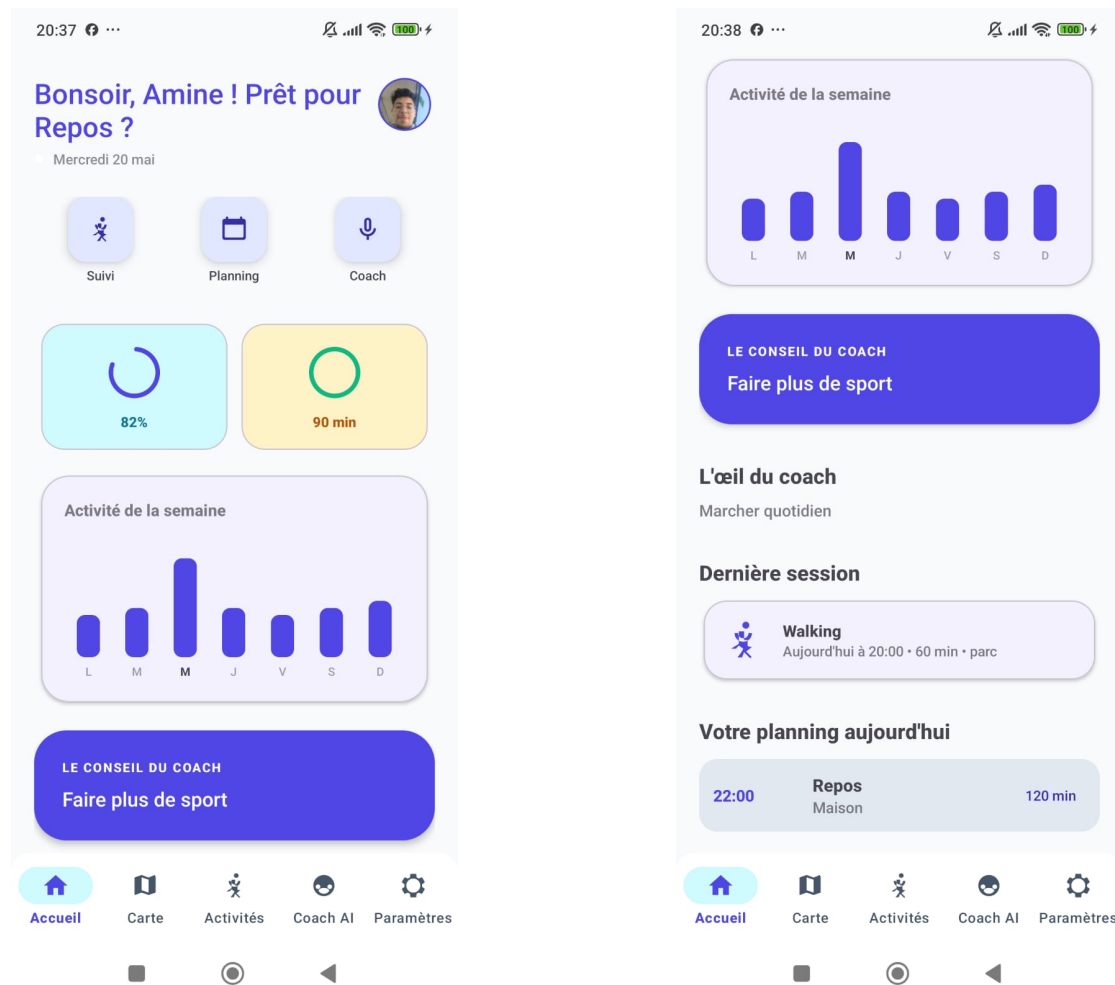
FIGURE 4.2 – Gestion du profil utilisateur — Consultation et édition

Description des écrans :

- **Profil utilisateur (Figure 4.2a)** : Affiche l'avatar de l'utilisateur (image personnalisée), puis une carte récapitulative avec les informations personnelles : Prénom (*Amine*), Nom (*benhadar*), Email (*abir@gmail.com*) et Date de naissance (*2004-09-19*). Deux actions sont disponibles : **"Edit Profile"** et **"Logout"** (en rouge).
- **Modification du profil (Figure 4.2b)** : Formulaire d'édition permettant de modifier le prénom, le nom et la date de naissance. Un sélecteur de date natif Android est intégré pour le champ *Birth Date*. Le bouton **"Save Changes"** envoie les mises à jour au backend FastAPI via un appel PUT.

4.1.3 Dashboard Principal (Accueil)

Le dashboard constitue la page d'entrée principale après authentification. Il agrège en un seul écran les indicateurs clés de santé, le graphique d'activité hebdomadaire, le conseil du Coach IA et un aperçu du planning du jour.



(a) Dashboard — Vue supérieure

(b) Dashboard — Vue inférieure (scroll)

FIGURE 4.3 – Dashboard SmartLife — Indicateurs de santé et résumé du jour

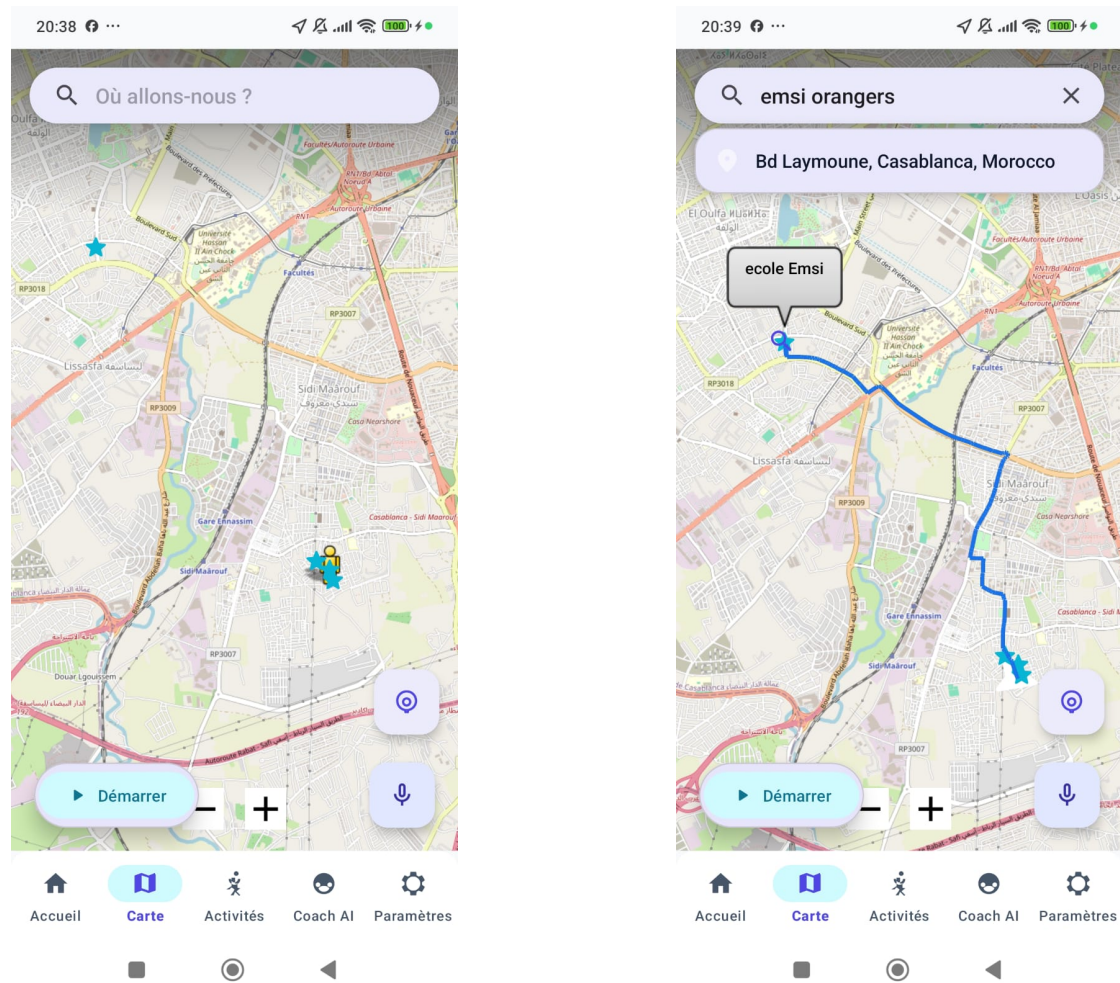
Description des composants du Dashboard :

- **En-tête personnalisée** : Message de salutation dynamique ("*Bonsoir, Amine ! Prêt pour Repos ?*") adapté à l'heure et à l'activité planifiée, avec la date du jour et la photo de profil de l'utilisateur.
- **Raccourcis rapides** : Trois boutons d'accès direct : **Suivi**, **Planning** et **Coach**.
- **Indicateurs circulaires** : Deux *CardViews* affichant :
 - Un cercle bleu indiquant le **score de discipline (82%)** calculé par le Coach IA.
 - Un cercle vert indiquant la **durée totale d'activité de la semaine (90 min)**.
- **Graphique d'activité hebdomadaire** : Histogramme à barres violettes représentant l'activité jour par jour (Lundi à Dimanche) avec mise en évidence du jour actuel (Mercredi — barre plus haute).

- **Conseil du Coach** : Bannière violette affichant "**LE CONSEIL DU COACH : Faire plus de sport**" généré automatiquement par l'API Groq en fonction du contexte.
- **Dernière session** : Carte récapitulative de la dernière activité enregistrée (*Walking* — *60 min* — *parc* — *20 :00*).
- **Planning du jour** : Affichage de la prochaine tâche planifiée (*Repos* — *Maison* — *22 :00* — *120 min*).

4.1.4 Module Suivi GPS et Cartographie OSMDroid

Ce module est le cœur sportif de SmartLife. Il offre une carte interactive basée sur **OpenStreetMap** via la bibliothèque OSMDroid, permettant de démarrer un suivi GPS, de visualiser sa position en temps réel, et de tracer le parcours effectué sous forme de polyline bleue.



(a) Carte — Position courante (Casablanca)

(b) Carte — Tracé du parcours (EMSI → destination)

FIGURE 4.4 – Module GPS — Cartographie OSMDroid et tracé de parcours en temps réel

Description des écrans :

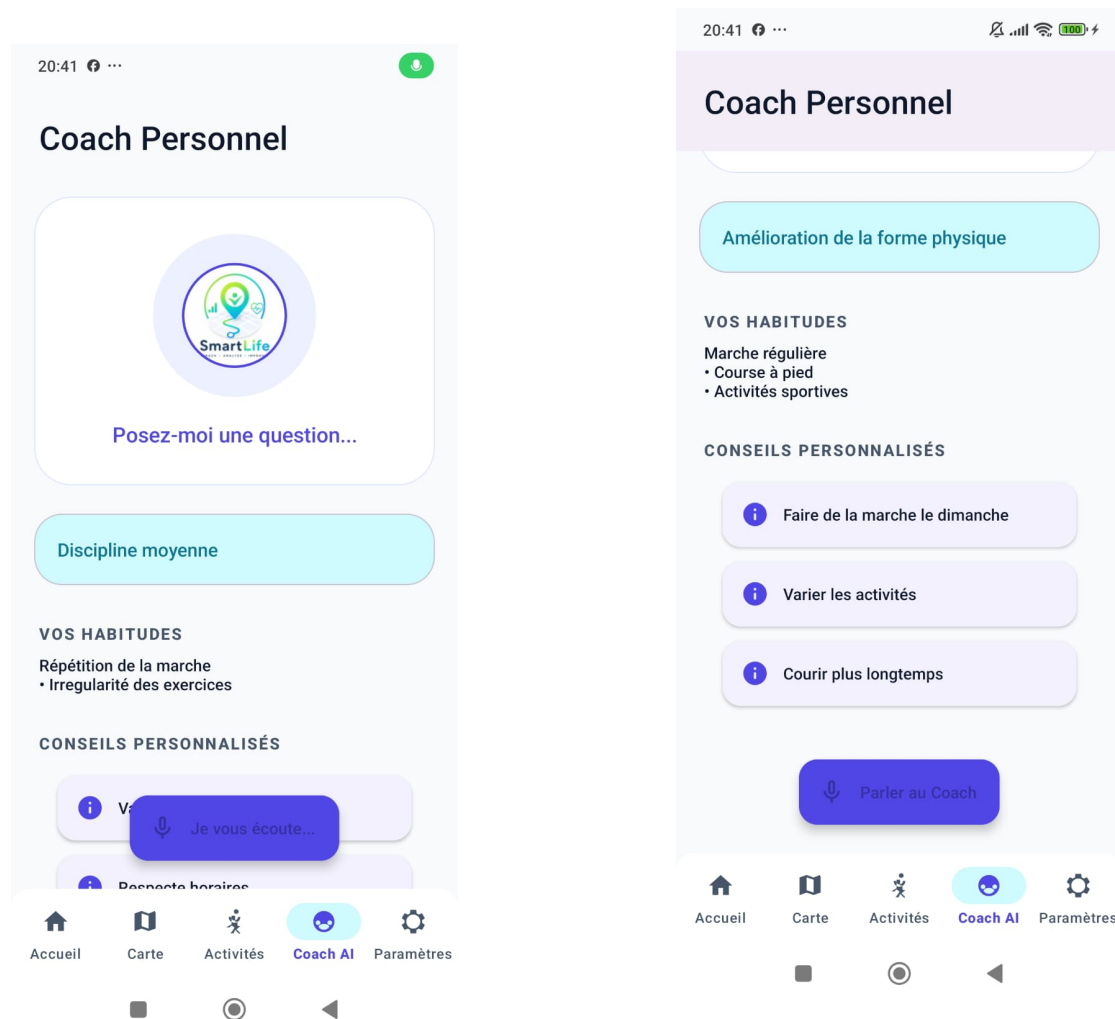
- **Carte principale (Figure 4.4a)** : Affiche la carte OpenStreetMap centrée sur la position GPS actuelle de l'utilisateur (quartier Sidi Maârouf, Casablanca). Une barre de recherche permet de saisir une destination ("Où allons-nous ?"). Le bouton "Démarrer" lance l'enregistrement GPS via le `GpsTrackingService`. Des contrôles de zoom (+/-) et un bouton de recentrage (📍) sont disponibles. Un bouton micro (🎤) permet d'interagir vocalement avec le Coach IA directement depuis la carte.
- **Tracé du parcours (Figure 4.4b)** : Après saisie de "emsi orangers" dans la barre de recherche, la carte affiche la suggestion "Bd Laymoune, Casablanca, Morocco" et trace un itinéraire en bleu depuis l'école EMSI (marquée "ecole Emsi") jusqu'à la position courante. La polyline bleue illustre le parcours enregistré.

TABLE 4.2 – Métriques collectées lors du suivi GPS

Métrique	Description et méthode de calcul	Unité
Distance parcourue	Somme des distances Haversine entre les coordonnées GPS consécutives	km
Durée de l'effort	Chronomètre depuis le démarrage jusqu'à l'arrêt du service	hh :mm :ss
Tracé cartographique	Polyline bleue dessinée sur la carte OSMDroid en temps réel	—
Localisation recherche	Géocodage de l'adresse saisie → affichage de l'itinéraire	Coordonnées GPS

4.1.5 Module Coach IA (Intelligence Artificielle)

Le Coach IA est la fonctionnalité différenciante de SmartLife. Il analyse le profil et l'historique d'activités de l'utilisateur pour générer des insights personnalisés, et répond aux questions vocales ou textuelles via le modèle LLaMA 3 (Groq API). Une animation *Pulse* de l'avatar indique que le micro est actif.



(a) Coach IA — Mode écoute vocale active

(b) Coach IA — Analyse et conseils personnalisés

FIGURE 4.5 – Module Coach IA — Interaction vocale et recommandations intelligentes

Description des écrans :

- **Mode écoute vocale (Figure 4.5a)** : L'écran affiche le logo SmartLife animé (avatar) avec le message d'invitation "*Posez-moi une question...*" et un badge de statut "**Discipline moyenne**". La section **VOS HABITUDES** liste les tendances détectées (*Répétition de la marche*, *Irrégularité des exercices*). En bas, un bandeau violet "**Je vous écoute...**" avec l'icône micro indique que le SpeechRecognizer est actif — c'est la phase STT.
- **Conseils personnalisés (Figure 4.5b)** : Après analyse, le Coach affiche l'objectif détecté ("**Amélioration de la forme physique**"), les habitudes positives (*Marche régulière*, *Course à pied*, *Activités sportives*) et trois **CONSEILS PERSONNALISÉS** générés par le LLM :
 - *Faire de la marche le dimanche*
 - *Varier les activités*

— *Courir plus longtemps*

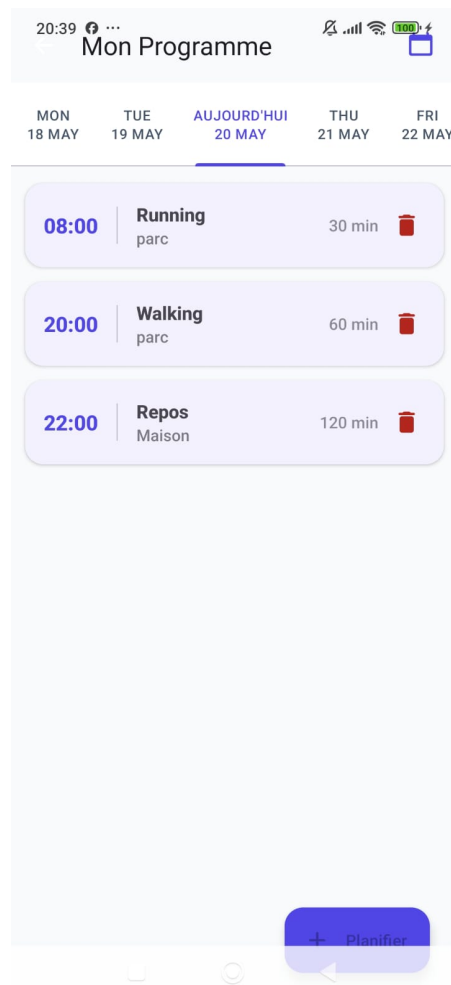
Un bouton **"Parler au Coach"** permet de relancer une interaction vocale.

TABLE 4.3 – Capacités du Coach IA implémentées dans SmartLife

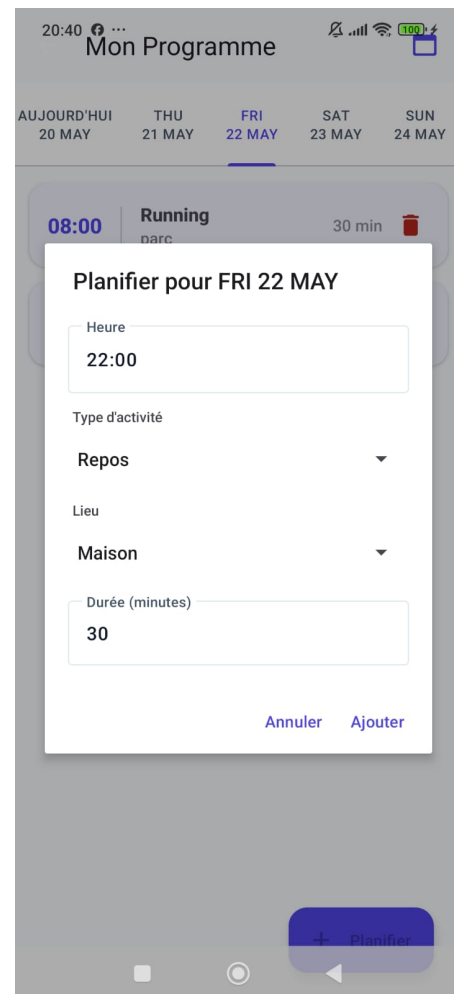
Capacité	Implémentation	Statut
Analyse des habitudes	Traitement de l'historique des activités MongoDB par le LLM	✓ OK
Score de discipline	Calcul automatique (0–100) basé sur la régularité des activités	✓ OK
Conseils personnalisés	3 recommandations contextuelles générées par Groq/LLaMA 3	✓ OK
Reconnaissance vocale STT	Android SpeechRecognizer avec animation d'écoute	✓ OK
Synthèse vocale TTS	TextToSpeech Android pour la lecture des réponses	✓ OK
Animation avatar	Animation Pulse du logo pendant l'écoute active	✓ OK

4.1.6 Module Calendrier et Planification

Le module de planification permet à l'utilisateur d'organiser ses séances sportives sur un calendrier hebdomadaire interactif. Les tâches planifiées sont synchronisées en temps réel avec le backend MongoDB via l'API FastAPI.



(a) Programme de la semaine — Jour actuel



(b) Formulaire d'ajout d'une tâche

FIGURE 4.6 – Module Calendrier — Planification hebdomadaire et ajout d'activités

Description des écrans :

- **Programme de la semaine (Figure 4.6a)** : Une barre de navigation horizontale permet de naviguer entre les jours de la semaine (MON 18 MAY → FRI 22 MAY), avec le jour actuel "**AUJOURD'HUI 20 MAY**" mis en évidence en bleu. Pour le jour sélectionné, toutes les tâches planifiées sont listées avec l'heure (bleue), le type d'activité, le lieu et la durée. Exemple : 08 :00 — *Running* — *parc* — 30 min, 20 :00 — *Walking* — *parc* — 60 min, 22 :00 — *Repos* — *Maison* — 120 min. Chaque tâche possède une icône poubelle rouge pour la suppression. Un FAB "+ **Planifier**" en bas à droite ouvre le formulaire d'ajout.
- **Formulaire de planification (Figure 4.6b)** : Dialog modal affiché sur un fond atténué. Titre dynamique "*Planifier pour FRI 22 MAY*". Les champs disponibles sont : Heure (champ texte), Type d'activité (dropdown : Repos, Running, Walking, Sport...), Lieu (dropdown : Maison, Parc, Salle...) et Durée en minutes. Deux boutons : **Annuler** et **Ajouter** (en bleu).

4.1.7 Journal d'Activités

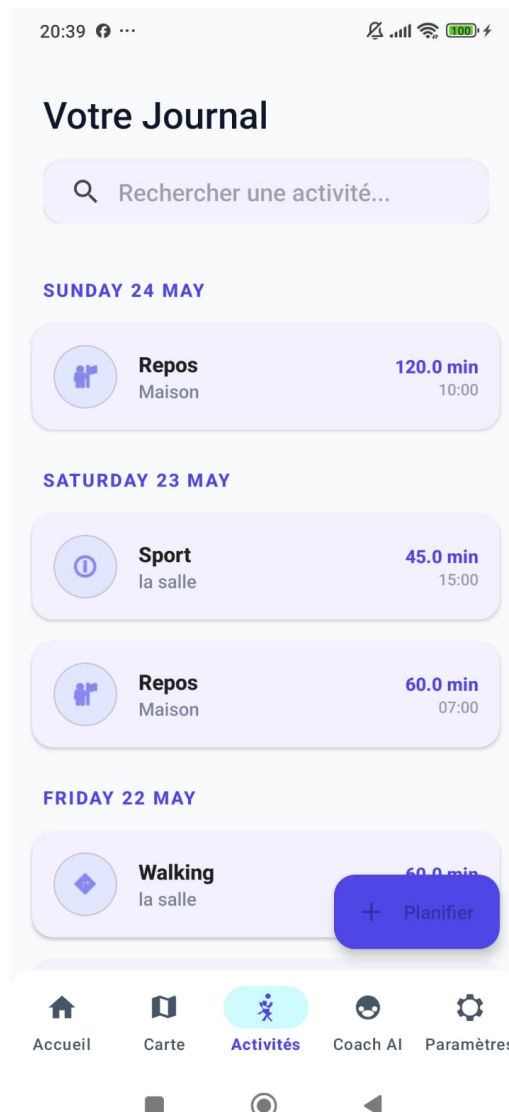


FIGURE 4.7 – Journal d'activités — Historique chronologique

Description : L'écran "Votre Journal" affiche l'historique complet des activités planifiées, trié par ordre antéchronologique (le plus récent en premier). Les activités sont regroupées par jour avec le libellé de la date en violet (*SUNDAY 24 MAY, SATURDAY 23 MAY...*). Chaque item affiche : l'icône de l'activité, le type (*Repos, Sport, Walking...*), le lieu, la durée en bleu et l'heure de début. Une barre de recherche en haut permet de filtrer les activités par mot-clé. Un FAB "+ Planifier" est disponible pour ajouter rapidement une nouvelle activité.

4.1.8 Paramètres et Personnalisation

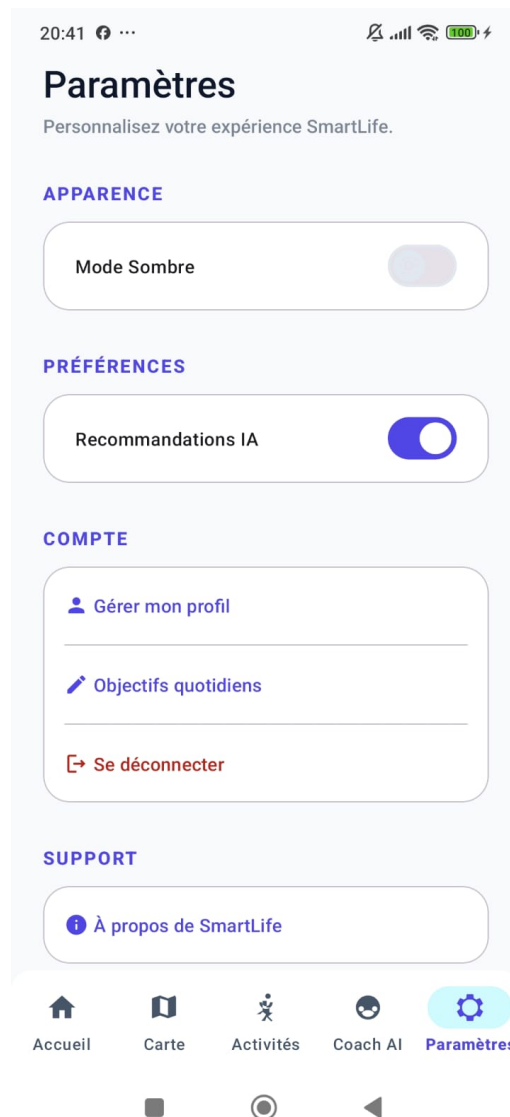


FIGURE 4.8 – Écran Paramètres — Personnalisation de l'expérience SmartLife

Description : L'écran "**Paramètres**" est organisé en quatre sections :

- **APPARENCE** : Toggle *Mode Sombre* (actuellement désactivé) pour basculer entre le thème clair et sombre.
- **PRÉFÉRENCES** : Toggle *Recommandations IA* (activé) pour activer/désactiver les conseils du Coach IA.
- **COMPTE** : Trois actions : *Gérer mon profil* (lien vers l'écran profil), *Objectifs quotidiens* (configuration des objectifs de santé) et *Se déconnecter* (en rouge).
- **SUPPORT** : Lien *À propos de SmartLife* avec les informations de version et mentions légales.

4.2 Synthèse des Interfaces Réalisées

TABLE 4.4 – Récapitulatif des écrans implémentés dans SmartLife

N°	Écran	Fonctionnalités principales	Statut
1	Login	Connexion Firebase, toggle password, lien inscription	✓ OK
2	Sign Up	Inscription complète, validation champs, création profil	✓ OK
3	Dashboard (Accueil)	Score santé, graphique semaine, conseil IA, planning jour	✓ OK
4	Carte GPS	OSMDroid, tracé parcours, recherche lieu, démarrer suivi	✓ OK
5	Coach IA	STT/TTS, analyse habitudes, conseils LLaMA 3, score discipline	✓ OK
6	Programme semaine	Calendrier hebdomadaire, CRUD tâches, navigation par jour	✓ OK
7	Journal d'activités	Historique trié, recherche, groupement par date	✓ OK
8	Profil utilisateur	Affichage infos, avatar, modifier, déconnexion	✓ OK
9	Édition du profil	Modification nom/prénom/date, sauvegarde via API PUT	✓ OK
10	Paramètres	Mode sombre, reco IA, gestion compte, support	✓ OK

4.3 Intégration Backend et Communication

TABLE 4.5 – Flux de communication Mobile — Backend FastAPI

Action	Source	Destination	Format & Détails
Création compte	Android	Firebase + FastAPI	POST /users/ → profil MongoDB créé
Récupération profil	Android	FastAPI	GET /users/{uid} → objet User JSON
Mise à jour profil	Android	FastAPI	PUT /users/{uid} → profil MongoDB mis à jour
Envoi activité GPS	Android	FastAPI	POST /activities/ {userId, type, distance, duration}
Chat Coach IA	Android	FastAPI→G	POST /ai/chat/ {message, context} → réponse LLM
Insights IA	Android	FastAPI→G	GET /ai/insights/{uid} → score + conseils JSON
Sync calendrier	Android	FastAPI	GET + POST /schedule/{uid} → WeeklySchedule

4.4 DevOps — Déploiement et Tests

4.4.1 Structure du Projet

TABLE 4.6 – Organisation des répertoires du projet SmartLife

Chemin	Contenu
SmartLife/app/src/main/	Code source Android (Kotlin)
.../java/.../ui/	Activités, Fragments, Adapters
.../java/.../viewmodel/	ViewModels (MVVM)
.../java/.../repository/	Repositories et sources de données
.../java/.../service/	GpsTrackingService (Foreground)
.../java/.../model/	Data classes Kotlin
.../res/layout/	Fichiers XML d'interface
SmartLife/backend/	Code source FastAPI (Python)
backend/main.py	Point d'entrée FastAPI + Uvicorn
backend/routers/	Modules de routes (users, activities, ai)
backend/ai_service.py	Interface avec l'API Groq
backend/.env	Clés API et configuration (non versionné)

4.4.2 Configuration et Déploiement

Le serveur FastAPI est configuré pour écouter sur 0.0.0.0:8000, permettant une liaison directe depuis l'appareil Android physique via le réseau Wi-Fi local. Toutes les captures d'écran présentées dans ce chapitre ont été prises sur un appareil réel lors des tests fonctionnels.

- **Backend** : Environnement virtuel Python (.venv), secrets dans .env (Groq API Key, MongoDB URI, Firebase credentials).
- **Firebase** : Fichier google-services.json côté Android, serviceAccountKey.json côté backend pour les notifications FCM.
- **MongoDB** : Instance locale ou cloud MongoDB Atlas selon l'environnement.
- **Tests** : Débogage croisé via le Logcat d'Android Studio et les journaux Uvicorn du backend.

Conclusion

La mise en œuvre de **SmartLife** a démontré l'efficacité et la cohérence du stack technologique choisi. Les dix écrans réalisés, illustrés par des captures d'écran prises sur

appareil réel, témoignent de la complétude fonctionnelle de l'application. L'intégration du Coach IA (Groq/LLaMA 3) avec reconnaissance et synthèse vocales, couplée au suivi GPS en temps réel sur carte OSMDroid, transforme SmartLife d'un simple outil de log en un véritable assistant de santé proactif et personnalisé.

Conclusion Générale et Perspectives

✓ Bilan du Projet SmartLife

Le projet **SmartLife** a permis de concevoir et de réaliser un assistant de santé connecté complet, alliant la puissance de l'intelligence artificielle à la précision de la géolocalisation mobile sur la plateforme Android.

Au cours de ce travail, plusieurs défis techniques majeurs ont été relevés :

- La gestion des **services d'arrière-plan** (Foreground Service) pour maintenir le suivi GPS sans interruption, tout en préservant la batterie.
- La **synchronisation temps réel** entre l'application mobile et le backend NoSQL via une API REST asynchrone.
- L'**intégration de modèles de langage** (LLM Groq/LLaMA 3) dans un flux mobile pour une expérience conversationnelle naturelle et contextuelle.
- La **gestion des permissions** Android (localisation, microphone) et leur intégration dans une UX fluide et non intrusive.

L'architecture **MVVM** a prouvé sa valeur en assurant une séparation claire des responsabilités, facilitant le débogage, les tests et l'évolution du code. Le choix de **FastAPI** + **MongoDB** côté backend a offert la flexibilité et les performances nécessaires pour un système orienté données en temps réel.

🚀 Perspectives et Évolutions Futures

- P1. Extension WearOS :** Porter l'application sur montres connectées WearOS pour récupérer des données biométriques (fréquence cardiaque, SpO2) en temps réel et enrichir les analyses du Coach IA.
- P2. Analyse Prédictive (ML) :** Exploiter l'historique des activités pour entraîner des modèles de machine learning capables de prédire les baisses de motivation et d'envoyer des encouragements proactifs ciblés.
- P3. Mode Hors-ligne complet :** Implémenter une base de données Room locale complète pour permettre le suivi et l'accès à l'historique sans connexion internet, avec synchronisation différée lors du retour en ligne.
- P4. Challenges et Gamification :** Introduire un système de défis sportifs

partagés entre utilisateurs, avec classements et récompenses, pour renforcer l'engagement sur le long terme.

P5. Intégration Wearables tiers : Connexion avec des appareils IoT de santé (tapis de course connecté, balance intelligente) via Bluetooth Low Energy (BLE) pour une collecte de données enrichie.

"SmartLife représente une étape vers une santé plus connectée et plus intelligente, mettant la technologie au service du bien-être quotidien de chacun."

Mohammed Amine BENHADAR

EMSI Orangers — Casablanca — Promotion 2025/2026